
Understanding Neural Network Binarization with Forward and Backward Proximal Quantizers

Yiwei Lu*

School of Computer Science
University of Waterloo
Vector Institute
yiwei.lu@uwaterloo.ca

Yaoliang Yu

School of Computer Science
University of Waterloo
Vector Institute
yaoliang.yu@uwaterloo.ca

Xinlin Li

Huawei Noah's Ark Lab
xinlin.li1@huawei.com

Vahid Partovi Nia

Huawei Noah's Ark Lab
vahid.partovinia@huawei.com

Abstract

In neural network binarization, BinaryConnect (BC) and its variants are considered the standard. These methods apply the sign function in their forward pass and their respective gradients are backpropagated to update the weights. However, the derivative of the sign function is zero whenever defined, which consequently freezes training. Therefore, implementations of BC (e.g., BNN) usually replace the derivative of sign in the backward computation with identity or other *approximate gradient* alternatives. Although such practice works well empirically, it is largely a heuristic or “training trick.” We aim at shedding some light on these training tricks from the optimization perspective. Building from existing theory on ProxConnect (PC, a generalization of BC), we (1) equip PC with *different* forward-backward quantizers and obtain ProxConnect++ (PC++) that includes existing binarization techniques as special cases; (2) derive a principled way to synthesize forward-backward quantizers with automatic theoretical guarantees; (3) illustrate our theory by proposing an enhanced binarization algorithm BNN++; (4) conduct image classification experiments on CNNs and vision transformers, and empirically verify that BNN++ generally achieves competitive results on binarizing these models.

1 Introduction

The recent success of numerous applications in machine learning is largely fueled by training big models with billions of parameters, e.g., GPTs in large language models [7, 8], on extremely large datasets. However, as such models continue to scale up, end-to-end training or even fine-tuning becomes prohibitively expensive, due to the heavy amount of computation, memory and storage required. Moreover, even after successful training, deploying these models on resource-limited devices or environments that require real-time inference still poses significant challenges.

A common way to tackle the above problems is through model compression, such as pruning [44, 47, 51], reusing attention [6], weight sharing [57], structured factorization [49], and network quantization [16, 30, 32, 38]. Among them, network quantization (i.e., replacing full-precision weights with lower-precision ones) is a popular approach. In this work we focus on an extreme case of network quantization: binarization, i.e., constraining a subset of the weights to be only binary (i.e., ± 1), with

*Work done during an internship at Huawei Noah's Ark Lab.